



Normalization

Normalization is a database design technique used to organize data properly so that:

- Data is **not repeated again and again**
- Data stays **accurate and consistent**
- Insert, update, and delete problems are avoided

What is Decomposition?

Decomposition means **breaking one big table into two or more smaller tables** to reduce redundancy and improve design.

Example (Original Table)

Student_Course

StudentID	StudentName	CourseID	CourseName
1	Ali	C1	DBMS
2	Sara	C1	DBMS

Here,

- **StudentName** depends only on **StudentID**
- **CourseName** depends only on **CourseID**

So we **decompose** the table.

After Decomposition

Student Table

StudentID	StudentName
1	Ali
2	Sara

Course Table

CourseID	CourseName
C1	DBMS

Lossless Decomposition

A decomposition is **lossless** if:

After splitting tables, we can join them back and get the exact original table.

Condition

Decomposition is **lossless** if:

- The **common attribute** is a **primary key** in at least one table

Example (Lossless)

Tables:

- Student(StudentID, StudentName)
- Enrollment(StudentID, CourseID)

Common attribute = **StudentID**

StudentID is a primary key → **Lossless**

👉 No data is lost when joining.

Lossy Decomposition

A decomposition is **lossy** if:

After joining decomposed tables, we cannot recover the original table correctly.

Example (Lossy)

Original Table:

R(A, B, C)

Decomposed into:

- R1(A, B)
- R2(B, C)

If **B is not a key**, joining R1 and R2 may produce:

- Extra rows
- Wrong combinations

👉 Information is lost or corrupted.

Intersection Method

This method does **not use actual data values**.

It uses **attributes and functional dependencies** only.

Rule

For a relation **R** decomposed into **R₁** and **R₂**:

The decomposition is lossless if

$(R_1 \cap R_2) \rightarrow R_1$ OR $(R_1 \cap R_2) \rightarrow R_2$

Meaning:

- The **common attributes (intersection)** must determine **all attributes of at least one table**
-

Example 1: Lossless Decomposition

Given Relation

$R(A, B, C)$

Functional Dependency

- $A \rightarrow B$

Decomposition

- $R_1(A, B)$
- $R_2(A, C)$

Step 1: Find Intersection

- $R_1 \cap R_2 = A$

Step 2: Check Dependency

- $A \rightarrow B$ (given)
- So **A determines all attributes of R_1**

 **Lossless decomposition**

Conclusion

Example 2: Lossy Decomposition

Given Relation

$R(A, B, C)$

Functional Dependency

- **No dependency involving the intersection**

Decomposition

- $R_1(A, B)$

- $R_2(B, C)$

Step 1: Find Intersection

- $R_1 \cap R_2 = B$

Step 2: Check Dependency

- Does $B \rightarrow A$? ❌
- Does $B \rightarrow C$? ❌

Conclusion

❌ Lossy decomposition

Because the common attribute **B** does **not determine** any table fully.

What is a Legal Instance?

A **legal instance** is a **table state (actual data)** that **follows all given functional dependencies**.

Legal Instance

A	B	C
1	X	5
1	X	9
2	Y	7

Why legal?

- $A = 1 \rightarrow B$ is always X ✓
- $A = 2 \rightarrow B$ is Y ✓

FD is not violated.

Illegal Instance ❌

A	B	C
1	X	5
1	Y	9

Why illegal?

- A = 1 gives **two different B values (X and Y)** ❌

FD $A \rightarrow B$ is violated.

Legal Instance + Decomposition (Connection 🔗)

When we say:

“A decomposition is lossless if every legal instance of R can be reconstructed”

It means:

- For **all valid data** that follows FDs
- Join of decomposed tables must give the original table

If even **one legal instance fails**, decomposition is **lossy**.

Definition of functional dependency

A functional dependency (FD) is a constraint between two sets of attributes in a relation, where the value of one set of attributes uniquely determines the value of another set of attributes.

It is written as:

$X \rightarrow Y$

Meaning:

If two rows have the same value of X, they must have the same value of Y.

Closure of a set of functional dependencies

The closure of a set of functional dependencies, denoted by F^+ , is the complete set of all functional dependencies that can be logically inferred from the given set of functional dependencies F .

In short:

F^+ = all FDs implied by

Example: Attribute Closure

Relation: **$R(A, B, C, D)$**

FDs:

- $A \rightarrow B$
- $B \rightarrow C$

Find **A^+**

Steps:

1. Start with **$A^+ = \{A\}$**
2. $A \rightarrow B \rightarrow$ add **B**
3. $B \rightarrow C \rightarrow$ add **C**

So:

$A^+ = \{A, B, C\}$

Keys and Functional Dependencies

Superkey

A **superkey** is a set of attributes that can **uniquely identify every tuple** in a table.

- If $K \rightarrow R$, it means **K determines all attributes of R**
- Example:

`ID → name, salary, dept_name, building, budget`

So, **ID is a superkey**

Candidate Key

A **candidate key** is a **minimal superkey**.

- It uniquely identifies tuples
- **No attribute can be removed** without losing uniqueness

Example:

- `{ID}` is a candidate key
- `{ID, name}` is a superkey but **not** a candidate key (extra attribute)

Keys and Functional Dependencies

- **K** is a **superkey** for relation schema **R** if and only if
 $K \rightarrow R$
- **K** is a **candidate key** for **R** if and only if
 - $K \rightarrow R$, and
 - there is **no proper subset** $\alpha \subset K$ such that $\alpha \rightarrow R$

Uses of Functional Dependencies

Functional dependencies are used to **define rules** about how data should behave in a relation.

1 To Test Whether a Relation is Legal

We use functional dependencies to check whether a **relation instance (table data)** is **legal or illegal**.

Key Idea

- If a relation **follows all functional dependencies in F**,
👉 we say **the relation r satisfies F**.

Example

FD:

- **ID → Name**

Legal instance ✅

ID	Name
1	Ali
1	Ali

Illegal instance ❌

ID	Name
1	Ali
1	Sara

2 "r satisfies F" vs "F holds on R" (Very Important ⚠️)

r satisfies F

- Talking about **one specific table instance**
- That table follows all FDs in F

F holds on R

- Talking about the **relation schema**
- **Every possible legal instance** of R must satisfy F

4 Important Note (Often Asked in Exams)

A specific relation instance may satisfy a functional dependency by chance, even if that dependency does **not always hold** on the schema.

Example

Relation: **Instructor(ID, Name)**

FD:

- **Name** → **ID** ❌ (generally false)

But a **specific instance** might be:

Name	ID
Rahim	101
Karim	102

Here:

- Each name is unique
- So **Name** → **ID** appears true

But:

- Two instructors can have the same name in real life
- So **Name** → **ID** does NOT hold on the schema

👉 This FD is satisfied **accidentally**, not logically.

Trivial Functional Dependencies

A functional dependency is called trivial if it is satisfied by every possible instance of a relation.

A functional dependency $\alpha \rightarrow \beta$ is **trivial**
if and only if $\beta \subseteq \alpha$

What is Lossless Decomposition?

A decomposition is **lossless** if:

After breaking a relation into smaller relations, we can join them back and get exactly the original relation, with no extra rows and no missing r

Example 1

Given

$R(A, B, C)$

$F = \{ A \rightarrow B, B \rightarrow C \}$

Case 1

$R_1(A, B)$ and $R_2(B, C)$

Step 1: Intersection

- $R_1 \cap R_2 = \{B\}$

Step 2: Check FD

- From $F: B \rightarrow C$
- So $B \rightarrow BC$ (means B determines all attributes of R_2)

✓ **Lossless decomposition**

Example 2

Decomposition

$R_1(A, B)$ and $R_2(A, C)$

Step 1: Intersection

- $R_1 \cap R_2 = \{A\}$

Step 2: Check FD

- From F: $A \rightarrow B$
- So $A \rightarrow AB$ (determines all of R_1)

✓ **Lossless decomposition**

Important Note from the Slide ⚠

These FD conditions are sufficient for lossless decomposition.

They are **necessary only if all constraints are functional dependencies**.

Dependency Preservation

A decomposition is dependency preserving if all functional dependencies can be enforced by checking the individual decomposed tables, without joining them.

In short:

👉 **No JOIN needed to check constraints = Dependency Preserving**

👉 **JOIN needed to check constraints = NOT Dependency Preserving**

Why is Dependency Preservation Important?

- Databases are updated frequently
- Checking constraints using **joins (Cartesian product)** is expensive 🧮
- We want constraints to be checked **table by table**

Given Example Explained Step-by-Step

Original Relation

dept_advisor(s_ID, i_ID, dept_name)

Functional Dependencies

1. $i_ID \rightarrow dept_name$
 2. $(s_ID, dept_name) \rightarrow i_ID$
-

Problem in Original Table

- dept_name is repeated many times
- Redundancy exists ✗

So we try to decompose.

Possible Decomposition

Instructor(i_ID, dept_name)

Advisor(s_ID, i_ID)

Now Check Dependency Preservation 🔍

FD 1: $i_ID \rightarrow dept_name$

- Can be checked in **Instructor** table alone ✓

FD 2: $(s_ID, dept_name) \rightarrow i_ID$

- s_ID is in **Advisor**
- dept_name is in **Instructor**
- i_ID is split across both

✗ Must JOIN tables to test this FD

Dependency-Preserving Decomposition ✓

Decompose into:

1 Instructor_Department

$R_1(i_ID, dept_name)$

FD preserved here:

- $i_ID \rightarrow dept_name$ ✓
-

2 Student_Dept_Advisor

$R_2(s_ID, dept_name, i_ID)$

FD preserved here:

- $(s_ID, dept_name) \rightarrow i_ID$ ✓
-

Boyce-Codd Normal Form

A relation schema R is in Boyce-Codd Normal Form (BCNF) with respect to a set of functional dependencies F if, for every functional dependency $\alpha \rightarrow \beta$ in F^+ , at least one of the following is true:

1. $\alpha \rightarrow \beta$ is trivial ($\beta \subseteq \alpha$), or
 2. α is a superkey of R
-

BCNF in Very Simple Words

| In BCNF, every determinant must be a superkey.

If some attribute determines others but is **not a key**,

👉 the table is **not in BCNF** ✗

Why BCNF is Stronger than 3NF

- 3NF allows some non-key determinants

- BCNF **does not allow them at all**

So BCNF removes **more redundancy**, but may **break dependency preservation**.

Given Example (NOT in BCNF)

Relation

`in_dep(ID, name, salary, dept_name, building, budget)`

Functional Dependencies

1. `ID → name, salary, dept_name`
2. `dept_name → building, budget`

Why This Relation is NOT in BCNF ❌

Consider FD:

- `dept_name → building, budget`

Here:

- `dept_name` is **not a superkey**
- But it determines other attributes

👉 Violates BCNF condition

BCNF Decomposition

Decompose into:

1 Instructor

`Instructor(ID, name, salary, dept_name)`

FD:

- `ID → name, salary, dept_name`

✓ `ID` is a superkey

✓ Instructor is in BCNF

2 Department

Department(dept_name, building, budget)

FD:

- dept_name → building, budget

✓ dept_name is a superkey

✓ Department is in BCNF

Is the Decomposition Lossless?

Yes ✓

- Common attribute = dept_name
 - dept_name → building, budget
 - Intersection determines **Department**
-

When Do We Decompose into BCNF?

If a relation R is not in BCNF, then there exists a functional dependency $\alpha \rightarrow \beta$ such that:

- α is **not a superkey**
- β is **not a subset of α**

This FD **violates BCNF** and forces decomposition.

BCNF Decomposition Rule (Very Important ★)

If $\alpha \rightarrow \beta$ violates BCNF, decompose R into:

1. $R_1 = (\alpha \cup \beta)$
2. $R_2 = R - (\beta - \alpha)$

This guarantees:

- **Lossless join**
 - Removal of BCNF violation
-

Apply the Rule to the Given Example

Original Relation

in_dep(ID, name, salary, dept_name, building, budget)

Violating FD

- **dept_name** → **building, budget**

Here:

- $\alpha = \text{dept_name}$
 - $\beta = \{\text{building, budget}\}$
-

Step 1: First Relation

$$R_1 = \alpha \cup \beta$$

👉 **Department**(dept_name, building, budget)

- **dept_name** is a key
 - Relation is in **BCNF** ✓
-

Step 2: Second Relation

$$R_2 = R - (\beta - \alpha)$$

- $\beta - \alpha = \{\text{building, budget}\}$
- Remove these from R

👉 **Instructor**(ID, name, dept_name, salary)

- **ID** is a key
 - Relation is in **BCNF** ✓
-

Given

$R(A, B, C)$

$F = \{ A \rightarrow B, B \rightarrow C \}$

Case 1: $R_1(A, B)$ and $R_2(B, C)$

Lossless-Join Check

- **$R_1 \cap R_2 = \{B\}$**
- From F : **$B \rightarrow C$**
- So **$B \rightarrow BC$** (i.e., B determines all attributes of R_2)

✓ **Lossless decomposition**

Dependency Preservation Check

- **$A \rightarrow B$** can be checked in **R_1**
- **$B \rightarrow C$** can be checked in **R_2**

✓ **Dependency preserving**

Case 2: $R_1(A, B)$ and $R_2(A, C)$

Lossless-Join Check

- **$R_1 \cap R_2 = \{A\}$**
- From F : **$A \rightarrow B$**
- So **$A \rightarrow AB$** (i.e., A determines all attributes of R_1)

✓ **Lossless decomposition**

Dependency Preservation Check

- **$A \rightarrow B$** can be checked in **R_1**
- **$B \rightarrow C$** cannot be checked in either R_1 or R_2 alone

(B and C never appear together)

✗ **Not dependency preserving**

(Requires joining R_1 and R_2 to check $B \rightarrow C$)

BCNF and Dependency Preservation

Key Statement

It is not always possible to achieve both BCNF and dependency preservation at the same time.

This is a classic DBMS trade-off.

Given Schema

`dept_advisor(s_ID, i_ID, dept_name)`

Functional Dependencies

1. $i_ID \rightarrow dept_name$
 2. $(s_ID, dept_name) \rightarrow i_ID$
-

Why **dept_advisor** is NOT in BCNF ✗

Look at the FD:

- $i_ID \rightarrow dept_name$

Here:

- **i_ID** is **not a superkey**
- But it determines another attribute

👉 Violates BCNF condition

Try BCNF Decomposition

To fix the BCNF violation, we decompose using:

- $i_ID \rightarrow dept_name$

Decomposition

1. **Instructor_Dept**(i_ID , $dept_name$)
2. **Advisor**(s_ID , i_ID)

Both relations are now:

✓ In BCNF

✓ Lossless

Now Check Dependency Preservation 🔍

FD 1: $i_ID \rightarrow dept_name$

- Preserved in **Instructor_Dept** ✓

FD 2: $(s_ID, dept_name) \rightarrow i_ID$

- s_ID is in **Advisor**
- $dept_name$ is in **Instructor_Dept**
- i_ID is split

✗ Cannot be checked without JOIN

Third Normal Form (3NF)

A relation schema **R** is in **Third Normal Form (3NF)** with respect to a set of functional dependencies **F** if for every functional dependency

$\alpha \rightarrow \beta$ in F^+ , **at least one** of the following holds:

1. $\alpha \rightarrow \beta$ is trivial ($\beta \subseteq \alpha$), or

2. α is a superkey of R, or
 3. Every attribute in $(\beta - \alpha)$ is part of some candidate key of R
(each attribute may belong to a different candidate key)
-

3NF in Very Simple Words

A table is in 3NF if non-key attributes do not depend on other non-key attributes,

unless those attributes are part of a candidate key.

This third condition is what **relaxes BCNF**.

Relationship Between BCNF and 3NF

- Every **BCNF** relation is automatically in 3NF 
- But a **3NF** relation may not be in BCNF 

Why?

- 3NF allows some non-key determinants to preserve dependencies
-

Example

Relation

dept_advisor(s_ID, i_ID, dept_name)

Functional Dependencies

1. $i_ID \rightarrow dept_name$
 2. $(s_ID, dept_name) \rightarrow i_ID$
-

Candidate Keys

- $\{s_ID, dept_name\}$

- {s_ID, i_ID}
-

Why **dept_advisor** is in 3NF 

Check FD 1: $i_ID \rightarrow dept_name$

- **i_ID** is not a superkey 
- But:
 - **dept_name** $\in \{s_ID, dept_name\}$ (a candidate key)

 Satisfies 3rd condition

Check FD 2: $(s_ID, dept_name) \rightarrow i_ID$

- LHS is a **superkey**

 Satisfies 2nd condition

 So the relation is in 3NF but not in BCNF

Redundancy in 3NF (Important)

3NF **does not completely eliminate redundancy.**

A relation is in **3NF** if:

1. It is already in **2NF**, and
2. For **every functional dependency** $X \rightarrow Y$, **at least one** is true:
 - **X** is a **superkey**, or
 - **Y** is a **prime attribute** (part of some candidate key)

No stories, no poetry. This rule decides everything.

Example Showing Redundancy

$R(J, K, L)$

$F = \{ JK \rightarrow L, L \rightarrow K \}$

This relation is in **3NF**, but:

Problems:

- **Repetition of data**
- **NULL values** may be required
(e.g., when L exists without a corresponding J)

So:

- 3NF is **not redundancy-free**
- BCNF is stricter

Comparison: BCNF vs 3NF

Aspect	BCNF	3NF
Redundancy	Minimal	Possible
Dependency preservation	Not guaranteed	Guaranteed
Lossless decomposition	Yes	Yes
Use of NULLs	Rare	Possible

Closure of a Set of Functional Dependencies

Given a set F of functional dependencies, the closure of F , denoted by F^+ , is the set of all functional dependencies that can be logically inferred from F .

In short:

F^+ contains every FD that must be true if F is true.

Simple Example

Given

$F = \{ A \rightarrow B, B \rightarrow C \}$

From this, we can infer:

- $A \rightarrow C$ (by transitivity)

So:

$F^+ = \{ A \rightarrow B, B \rightarrow C, A \rightarrow C, \text{ (and all trivial FDs) } \}$

We can compute F^+ , the closure of a set of functional dependencies F , by repeatedly applying Armstrong's Axioms:

- **Reflexivity rule:** If $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
- **Augmentation rule:** If $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$
- **Transitivity rule:** If $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$

Additional Rules of Functional Dependencies

- **Union rule:**
If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds.
- **Decomposition rule:**
If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds.
- **Pseudotransitivity rule:**
If $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds.
- The above rules **can be inferred from Armstrong's axioms.**

Canonical Cover

◆ Definition

A **canonical cover (F_c)** of a set of functional dependencies F is a **minimal set of functional dependencies** such that:

1. $F_c^+ = F^+$ (same closure)
2. No functional dependency in F_c has:
 - extraneous attributes on the left-hand side

- more than one attribute on the right-hand side
3. No functional dependency in F_c is redundant

Example

Given:

$$F = \{ AB \rightarrow C, A \rightarrow C \}$$

Check if **B** is extraneous in **$AB \rightarrow C$** :

- Compute $A^+ = \{A, C\}$
- Since $C \in A^+$, **B is extraneous**

So:

$$AB \rightarrow C \quad \rightarrow A \rightarrow C$$